

Function Calling 的难点，不是让模型“会调用函数”，而是让模型在真实业务环境中**安全、稳定、可控、可追责**地调用函数。很多系统 Demo 阶段看起来能跑，一到生产就出问题，通常不是模型不会用工具，而是工具设计、schema 约束、权限控制、错误处理、日志审计和业务接入没有做好。

真正的工程能力，体现在能不能提前把风险封住。

1. 工具怎么设计：避免“万能工具”

一个常见坏设计，是把内部函数原样暴露给模型，比如：

```
execute_sql(sql)
send_request(url, body)
run_python(code)
delete_file(path)
```

这类工具的问题是能力过大，模型一旦误用，就可能查错库、删错文件、发错请求，甚至访问不该访问的数据。

具体操作中，应把工具设计成**窄工具、业务工具、可验证工具**。比如不要给模型 `execute_sql(sql)`，而是给：

```
query_customer_orders(customer_id, start_date, end_date)
get_product_revenue(product_line, period)
search_policy_docs(query, top_k)
create_refund_draft(order_id, reason, amount)
```

也就是说，模型看到的不是底层技术能力，而是被封装过的业务动作。工具越窄，误用空间越小；**工具越贴近业务语义，模型越容易正确选择。**

为了避免工具滥用，可以给每个工具加四个字段：适用场景、不适用场景、输入限制、输出格式。例如：

```
{
  "name": "search_policy_docs",
  "description": "用于检索公司制度、流程、合规政策。不得用于查询客户个人信息或交易数据。",
  "risk_level": "low",
  "requires_human_confirmation": false
}
```

这样工具本身就变成了系统边界的一部分。

2. schema 怎么约束：避免参数随意生成

Function Call 的 schema 不能只写字段名。很多系统出问题，是因为 schema 太松，模型虽然生成了 JSON，但参数含义不清，边界不明。

比如这个 schema 就很危险：

```
{
  "query": "string",
  "options": "object"
}
```

模型可以在 options 里塞任何东西，后端很难校验。

更好的方式是把字段拆清楚：

```
{
  "type": "object",
  "properties": {
    "query": {
      "type": "string",
      "minLength": 2,
      "maxLength": 100
    },
    "market": {
      "type": "string",
      "enum": ["A_SHARE", "HK", "US"]
    },
    "top_k": {
      "type": "integer",
      "minimum": 1,
      "maximum": 10
    }
  },
  "required": ["query", "market", "top_k"],
  "additionalProperties": false
}
```

具体操作中，要做到三层校验。

第一层是模型侧 schema，让模型尽量生成正确参数。

第二层是服务端 schema validation，模型生成后必须重新校验。

第三层是业务规则校验，比如日期不能超过当前日期，金额不能为负，查询范围不能超过用户权限。

不要相信模型“看起来生成对了”。Function Call 的参数必须像外部用户输入一样处理：默认不可信，必须校验。

3. 哪些工具暴露给模型：避免暴露内部系统能力

不是系统里有什么能力，就应该给模型什么能力。工具暴露要遵循“最小权限原则”。

可以自动暴露的，一般是只读、低风险、可重复执行的工具，例如：

```
search_docs
read_ticket
get_order_status
retrieve_chunks
summarize_table
```

谨慎暴露的是会改变状态的工具，例如：

```
update_customer_note
create_ticket
submit_form
send_notification
```

原则上不应直接暴露的是高危工具，例如：

```
execute_sql
run_shell
send_http_request
delete_record
transfer_money
change_permission
```

具体操作中，可以建立一个 Tool Registry，每个工具都登记以下信息：

```
{
  "tool_name": "create_refund_request",
  "risk_level": "medium",
  "read_or_write": "write",
  "allowed_roles": ["customer_service_manager"],
  "requires_approval": true,
  "max_amount": 5000,
```

```
"audit_required": true
}
```

然后每次模型想调用工具时，不是直接执行，而是先经过 Tool Gateway 判断：这个用户有没有权限？这个动作风险等级是多少？是否需要人工确认？参数是否超过限制？

模型只能“提出调用意图”，系统决定能不能执行。

4. 哪些动作必须人工确认：避免模型直接改变现实世界

Function Calling 一旦连接真实业务系统，就必须区分“生成建议”和“执行动作”。

凡是会对外部世界产生影响、不可轻易回滚、涉及资金或权益、影响客户或监管结果的动作，都应人工确认。

例如：

```
发送正式邮件
提交审批
删除数据
修改客户资料
触发退款
发起交易
关闭工单
对外发布公告
调用生产环境接口
```

具体操作中，可以把动作分成三级。

低风险动作：自动执行。比如检索文档、读取订单状态、生成摘要。

中风险动作：先生成草稿或待办。比如生成邮件草稿、创建待审批工单。

高风险动作：必须人工审批后执行。比如发送邮件、退款、删除、批量修改。

一个好的实践是：让模型永远先 draft，不直接 submit。

例如模型不要直接调用：

```
send_email(to, subject, body)
```

而是先调用：

```
create_email_draft(to, subject, body)
```

然后由用户确认后，再由系统调用真实发送接口。

5. 工具结果如何返回：避免把原始结果塞回模型

很多系统会把 API 返回结果原样塞给模型，这会带来三个问题：内容太长、字段太乱、来源不清。

工具结果应该经过结构化整理后再返回。推荐格式是：

```
{
  "status": "success",
  "summary": "找到 5 条相关制度条款，其中 2 条与审批权限直接相关。",
  "data": [],
  "evidence": [
    {
      "source_id": "doc_001",
      "title": "费用审批管理办法",
      "chunk_id": "chunk_12",
      "quote": "单笔超过 5 万元需部门负责人审批"
    }
  ],
  "warnings": [],
  "next_actions": ["可以继续查询历史审批案例"]
}
```

具体操作中，工具返回要做到四点。

第一，只返回模型需要的信息，不返回全量原始数据。

第二，重要结论必须带来源，例如 doc_id、row_id、record_id。

第三，返回结果要标明状态：成功、部分成功、失败、空结果。

第四，对敏感字段做脱敏，例如手机号、身份证号、客户账户。

模型最终回答时，只能引用工具返回中的事实，不能凭空补全。

6. 错误如何降级：避免失败后编造答案

Function Call 一定会失败。生产系统中常见错误包括：参数校验失败、接口超时、权限不足、结果为空、限流、工具异常、返回格式不符合预期。

错误不能只返回一句：

```
error
```

而要返回结构化错误：

```
{
  "status": "failed",
  "error_type": "PERMISSION_DENIED",
  "message": "当前用户无权查询该客户数据",
  "retryable": false,
  "suggested_action": "请申请客户数据访问权限或改用脱敏查询工具"
}
```

具体操作中，不同错误要有不同处理。

Schema 错误：允许模型修正参数一次，不能无限重试。

超时错误：缩小查询范围，或使用缓存结果。

权限不足：停止执行，提示需要授权。

结果为空：明确说明没有查到，不要编造。

限流错误：降频、排队或返回部分结果。

工具异常：进入 fallback，而不是继续调用同一个工具。

要给 Agent 设置最大重试次数，例如同一个工具最多重试 2 次。如果两次都失败，就停止，并告诉用户哪些部分完成了，哪些部分没完成。

7. 多工具调用如何编排：避免模型自由乱跑

复杂任务往往需要多个工具。比如生成一份客户分析报告，可能要查客户信息、交易数据、产品持仓、政策限制、历史沟通记录。不能完全让模型自由决定调用顺序，否则容易漏步骤、重复调用或陷入循环。

具体操作中，可以采用“状态机（指提前定好「步骤节点 + 流转规则」的任务流程框架） + 模型决策”的方式。

例如任务流程可以被定义为：

理解需求

- 查询客户基础信息
- 查询交易与持仓数据
- 检索相关政策
- 汇总风险点
- 生成报告草稿
- 人工确认

模型可以在每一步内部做判断，但不能跳过关键步骤。

还可以给每个任务定义 checklist：

```
{
  "task": "generate_customer_report",
  "required_steps": [
    "get_customer_profile",
    "get_transaction_summary",
    "retrieve_policy_docs",
    "generate_report"
  ],
  "max_tool_calls": 8,
  "stop_condition": "report_generated_with_evidence"
}
```

这样可以避免 Agent 调了一堆工具，却没有真正完成任务。

对于多工具任务，尤其要做三件事：限制最大调用次数，记录每一步状态，判断是否有新信息增量。如果连续两次调用没有产生新信息，就应停止。

8. 如何记录日志和审计：避免问题发生后查不清

只要模型能调用工具，就必须有完整日志。否则一旦出错，你不知道是模型理解错了、schema 失效了、权限没拦住，还是工具返回了错误数据。

具体操作中，每一次 Function Call 至少记录：

```
{
  "request_id": "req_001",
  "user_id": "user_123",
  "agent_version": "v1.2.0",
  "model": "xxx",
  "tool_name": "query_customer_orders",
  "tool_args": {},
  "schema_validation": "passed",
  "permission_check": "passed",
  "risk_level": "medium",
  "human_confirmation": "required",
  "tool_status": "success",
  "latency_ms": 1200,
  "token_usage": 3500,
  "cost": 0.02,
  "final_answer_id": "ans_001"
}
```

审计日志还要关注三个问题。

第一，谁触发了动作。

第二，模型为什么选择这个工具。

第三，这次工具调用影响了哪些业务对象。

对于高风险动作，要额外记录人工确认人、确认时间、确认内容和执行结果。这样系统才能在生产环境中被追责、复盘和优化。

9. 如何控制成本和上下文：避免工具越多越贵

Function Calling 会增加成本，因为工具定义要进入上下文，工具结果也要进入上下文，多轮调用还会继续累积 token。

具体操作中，可以从四个环节控制。

第一，不要一次暴露全部工具。根据当前任务动态选择工具。例如用户只是问制度，不需要暴露订单、邮件、数据库工具。

第二，工具描述要短而准，不要把说明书塞进 tools。

第三，工具结果要摘要化，长文档先检索，再返回 top_k 片段。

第四，设置上下文预算，例如最多保留最近 5 轮、最多返回 10 条证据、单次工具结果不超过 3000 字。

还有一个重要原则：**确定性工作不要交给模型。**

排序、过滤、去重、计算、权限判断、schema 校验、字段映射，都应该由代码完成。

模型只负责理解意图、选择工具、综合解释。这样既省成本，也更稳定。

10. 如何接入 RAG、数据库、工作流和业务系统：避免模型直连核心系统

Function Calling 的价值，是把模型接入企业已有系统。但接入方式不能是“模型直接访问核心系统”。

接入 RAG 时，应提供受控检索工具：

```
retrieve_documents(query, filters, top_k)
```

返回结果要包含：

```
{
  "doc_id": "制度文件编号",
  "chunk_id": "片段编号",
  "title": "文件标题",
  "score": 0.87,
  "text": "相关片段",
```



```
"updated_at": "2026-05-01"
}
```

这样模型回答时可以引用来源，避免把旧文件、弱相关文件当成事实依据。

接入数据库时，不要直接给模型 SQL 执行权限。更好的方式是提供固定查询工具：

```
get_customer_orders(customer_id, date_range)
get_revenue_summary(department, period)
get_risk_events(customer_id, period)
```

如果确实需要模型生成 SQL，也必须经过 SQL 白名单、只读权限、字段权限、行数限制和人工审核。

接入工作流时，模型不应直接改变流程状态，而是创建“待办”或“审批单”。例如：

```
create_approval_request(type, payload)
create_ticket(category, priority, description)
```

真正的状态流转由工作流系统控制。

接入业务系统时，最好通过 Tool Gateway 统一治理：

```
模型
→ Tool Schema 校验
→ 权限检查
→ 风险分级
→ 人工确认
→ 调用业务 API
→ 结果脱敏与摘要
→ 日志审计
→ 返回模型
```

这样模型不直接面对数据库、API、权限系统和业务流程，而是在一个受控通道里工作。

结语

Function Calling 看起来只是“模型调用函数”，但生产级系统真正考验的是：你有没有把模型的不确定性关进工程边界里。

坏的 Function Calling 是：

模型想调什么就调什么
模型生成什么参数就执行什么
工具返回什么就塞回模型
失败了就让模型继续猜

好的 Function Calling 是：

工具被精心封装
schema 被严格约束
权限由系统裁决
高风险动作人工确认
工具结果结构化返回
错误有明确降级路径
多工具调用有状态机编排
所有调用可日志、可审计、可回放
成本和上下文被预算管理
业务系统通过网关接入

所以，Function Calling 的工程能力不在于“会写几个 tools”，而在于能不能回答这些问题：

这个工具为什么要暴露给模型？
模型可以在什么条件下调用它？
参数错了怎么办？
权限不够怎么办？
工具失败怎么办？
结果有没有证据来源？
调用成本是否可控？
高风险动作是否有人确认？
出了问题能不能追溯？
这次调用是否真的推动了业务任务完成？

能回答这些问题，Function Calling 才不是 Demo，而是可以进入生产环境的 Agent 工程能力。